Turing Challenge / TechSolutions

Diseño de arquitectura para una API B2B de visión artificial

Atilio Camargo Moreira | 23 de octubre de 2026 | Documento para discusión técnica

Mi propuesta parte de una API asíncrona multi-tenant con Azure API Management, procesamiento por etapas desacoplado con Service Bus, PostgreSQL/PostGIS para el catastro de 150 millones de parcelas, inferencia GPU gestionada y almacenamiento privado de resultados. La solución prioriza trazabilidad, control de costes y crecimiento medible, evitando dimensionar las GPUs con una hipótesis no validada.

1. Enfoque y decisiones de partida

La petición combina tres tareas con tiempos medios declarados: preparación 2 s, inferencia 10 s y lógica posterior 3 s. La latencia base sería de aproximadamente 15 s sin colas, variabilidad ni fallos. No bloquearía una conexión HTTP esperando todo ese proceso como contrato principal: respondería 202 Accepted con jobId, permitiría GET /v1/jobs/{id} y, opcionalmente, enviaría un webhook firmado.

La aceptación del trabajo y el resultado final tienen SLA diferentes. El cliente empresarial obtiene identidad propia, cupos, reportes de consumo y acceso únicamente a sus resultados. Las etapas se recuperan de fallos sin repetir llamadas externas cuando ya existe un resultado válido.

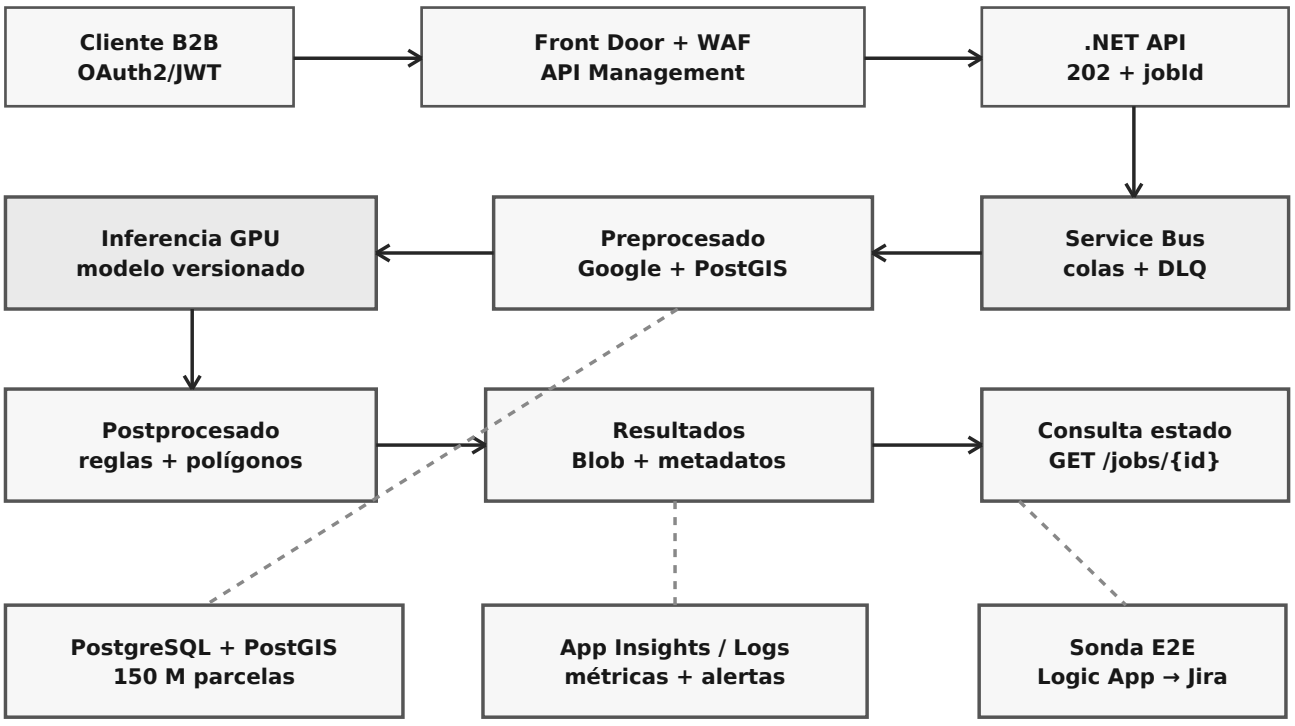
Decisión	Beneficio para el negocio
API asíncrona y mensajería	Absorber ráfagas sin bloquear clientes ni comprar capacidad GPU para cada pico instantáneo.
PostGIS para catastro	Búsquedas espaciales con índices y capacidad de explicar / optimizar planes de consulta.
GPU escalable con benchmark	Pagar por capacidad real y evitar prometer latencias sin medir el modelo.
Quota + registro de uso	Gobernar contratos B2B, prevenir abuso y producir facturación auditable.

2. Dimensionamiento: primero los números

Indicador	Cálculo / implicación
Tiempo de servicio nominal	2 + 10 + 3 = 15 s por petición (sin cola).
Concurrencia teórica a 50 RPS	Ley de Little: 50 × 15 = 750 solicitudes en curso si la entrada es sostenida.
Inferencia sin paralelismo interno	50 × 10 = 500 inferencias simultáneas requeridas a régimen permanente. NO equivale automáticamente a 500 GPUs.
50 RPS continuos	180.000 peticiones/hora; 2 M anuales se consumen en 11,1 horas a ese ritmo.
1 RPS continuo durante un año	31,5 M peticiones/año: contradice el volumen anual comunicado.
Datos de resultados brutos	2 M × (405 KB a 1.030 KB) ≈ 0,81–2,06 TB/año (un JSON y una imagen por petición).

Premisa obligatoria: interpreto 50 RPS como ráfagas durante marzo-mayo, no carga mantenida. La duración de las ráfagas, la llegada simultánea, la distribución horaria, los reintentos y el tiempo máximo permitido en cola deben confirmarse con TechSolutions antes de comprar capacidad.

3. Arquitectura de referencia



Flujo principal simplificado. Fases desacopladas con estado persistente, reintentos e idempotencia.

Flujo operativo y contrato

1) Entra ID emite token OAuth2 client credentials con tenantId/scopes. Front Door/WAF protege la entrada y APIM valida JWT, aplica rate-limit, cuota y correlationId. 2) La API valida idempotency-key, registra el job en una transacción y publica a la cola mediante outbox para evitar estados huérfanos. 3) Preprocesamiento resuelve dirección, consulta PostGIS, llama a Google Maps con timeouts, retry limitado y caché permitida por licencia, y persiste sus artefactos. 4) Inferencia GPU procesa el artefacto de entrada de versión conocida. 5) Postprocesado transforma máscaras a polígonos, simplifica vértices, calcula superficies y flags; almacena salida JSON e imagen. 6) El cliente obtiene resultado o error tipificado mediante polling o webhook.

Máquina de estados: RECEIVED → PREPROCESSING → READY_FOR_INFERENCE → INFERENCING → POSTPROCESSING → COMPLETED; FAILED_RETRYABLE / FAILED_FINAL / CANCELLED. Cada etapa registra versión de código, parámetros, timestamps, intento, coste estimado y URI de artefactos. No se deben guardar imágenes ni datos personales completos en logs.

Modelo de API propuesto

Operación	Contrato mínimo
POST /v1/evaluations	202 {jobId, statusUrl, submittedAt}; Idempotency-Key; 400/401/403/429/503.
GET /v1/evaluations/{jobId}	200 {status, progress, resultUrl?, error?}; siempre filtrado por tenant.
GET /v1/evaluations/{jobId}/result	200 resultado JSON o URL temporal de lectura autorizada; 409 si no está listo.
GET /v1/usage	Volumen admitido, completado, fallido y facturable por periodo/tenant.

4. Catastro, datos y almacenamiento

Azure Database for PostgreSQL Flexible Server + PostGIS. Guardaría geometrías en SRID normalizado, con índices espaciales GiST, índices B-tree sobre claves catastrales, estadísticas actualizadas y particionado solo si pruebas reales lo justifican. Las búsquedas por coordenada usarán ST_Intersects/ST_Covers según la semántica acordada. Validaría topología, geometrias inválidas y cambios de versión. Para 150 M de filas: carga masiva por lotes, staging, COPY, pruebas de índices, EXPLAIN ANALYZE, pruebas de p95 y política de VACUUM/analyze. Separaría datos operacionales de resultados y del ledger de consumo.

Blob Storage privado: JSON e imágenes en contenedores con claves lógicas tenant/job, cifrado, acceso por identidad administrada, endpoints privados, lifecycle y retención configurable. SAS de corta duración solo cuando haga falta descarga directa; preferir acceso mediado/autorizado. Habilitar soft delete/versionado según política de recuperación, evitando multiplicar retención sin presupuesto.

Licencias y privacidad: la reutilización y retención de imágenes y datos de Google está condicionada a sus términos concretos y al acuerdo empresarial. No asumir autorización de caché permanente. Definir clasificación, residencia UE, DPA y políticas de borrado por cliente.

5. Escalabilidad e inferencia de IA

Separaría autoscaling de CPU (pre y post) y GPU (inferencia). Los workers de CPU escalan por backlog y antigüedad de mensajes mediante KEDA; el límite superior protege PostGIS, Google y presupuesto. La GPU puede alojarse en Azure Container Apps serverless GPU (T4/A100 según región y cuota) si el contenedor y el tiempo de arranque lo permiten. La alternativa es Azure Machine Learning managed online endpoint, adecuada si el gobierno de modelos, versionado y MLOps lo justifican. No usaría AKS por defecto por coste operativo adicional.

Parámetro	Regla de decisión
Throughput de GPU	Medir q = inferencias/s/réplica con imágenes reales, p50/p95/p99 y batch. Réplicas ≈ ceil(RPS objetivo/q) + margen; sujeto a cuota.
SLA de cola	Medir oldest-message-age y tiempo total por percentiles; alertar por incumplimiento de promesa, no solo por CPU.
Ráfagas de entrada	La cola amortigua el pico; dimensionar por duración de ráfaga y plazo máximo de finalización.
Protección externa	Bulkheads y límite de concurrencia por API externa; retry con backoff/jitter; circuit breaker; degradación explícita.
Disponibilidad GPU	Solicitar cuota regional con antelación y preparar modalidad alternativa; test de cold start en entorno objetivo.

Ejemplo comprobable de backlog

Si entran 50 RPS durante 10 minutos y la solución procesa 10 RPS sostenidos, se acumulan $(50-10) \times 600 = 24.000$ trabajos. Tras el pico, si la nueva entrada es 1 RPS, el excedente se drena a 9 RPS: unos 44,4 minutos adicionales. Este cálculo obliga a negociar la latencia final deseada y el coste de capacidad de reserva.

6. Seguridad y modelo multi-tenant

Capa	Medidas principales
Identidad y API	Entra ID (OAuth2/JWT), scopes y roles por producto, APIM en tier compatible con cuotas por tenant, rotación de credenciales y mTLS opcional.
Aislamiento	TenantId validado en servidor, filtro en toda consulta y ruta de Blob, pruebas de acceso cruzado; RLS PostgreSQL donde sea apropiado.
Red y secretos	Front Door Premium/WAF; ingress interno cuando encaje; private endpoints para PostgreSQL/Storage/Key Vault; managed identities; egress controlado a terceros.
Amenazas	WAF y protección DDoS en el perímetro; rate-limit y cuotas, límite de payloads, validation estricta, escaneo de imágenes/dependencias.
Gobernanza	RBAC mínimo, PIM, Azure Policy, Defender for Cloud, presupuesto por etiquetas, logs de auditoría y CI/CD con federated credentials.

Matiz importante: las quotas de APIM no sustituyen un ledger de facturación. El ledger almacenará evento único por job y tenant, política de cobro y estado (admitido, completado, rechazado, reintentado). Las políticas y su disponibilidad dependen del tier APIM elegido; validarlo antes de estimar el coste.

7. Observabilidad, pruebas sintéticas y respuesta a incidentes

Instrumentaría .NET con OpenTelemetry y Azure Monitor/Application Insights, propagando traceId/jobId/tenantId pseudonimizado por Service Bus. Mediría admisión p95, E2E p50/p95/p99, tasa de 429, edad del mensaje más antiguo, backlog por etapa, DLQ, coste Google, tiempos por API externa, throughput GPU, error por versión del modelo y acierto funcional de la sonda.

Sonda E2E: Azure Function con timer lanza una entrada sintética fija, espera/polleará hasta timeout y compara el resultado con tolerancias declaradas (no comparación binaria de flotantes/geometría). Publica métrica custom y crea incidencia en Jira mediante Logic Apps/Jira REST con deduplicación por regla + ventana temporal. Ejecuta recuperación y cierra o actualiza la incidencia según política. Mantendría también un test HTTP ligero de disponibilidad en Application Insights. No debe abrir un ticket nuevo en cada ejecución mientras persista el mismo incidente.

Alerta	Umbral inicial y actuación
Disponibilidad externa	2 o 3 fallos sintéticos consecutivos o timeout → severidad alta y Jira.
Cola	Oldest-message-age mayor que el objetivo contratado por 5 min → escalar/investigar GPU.
Errores	Tasa 5xx mayor de 2% durante 5 min con volumen mínimo → guardia.
Latencia	p95 admisión > 500 ms o p95 E2E > SLO acordado → investigar etapa lenta.
Costes	Budget 70/90/100%, coste por evaluación por encima de objetivo → FinOps.
Datos	DLQ > 0 sostenida, fallos de backup/restauración y errores de consistencia → incidente.

8. Calidad, CI/CD y operación

Azure DevOps con pipelines YAML. Pull request: formato/linters, SAST, análisis de secretos, SCA, tests unitarios de reglas geográficas, contract tests y validación Bicep. Build: artefacto firmado, SBOM, imagen inmutable en ACR y control de vulnerabilidades. Release: entorno dev → test → preprod → prod con aprobaciones, OIDC/workload identity federation (sin secretos persistentes), IaC y comprobación de drift. En producción usaría revisión/canary con métricas de error y latencia; rollback a imagen/modelo anterior y feature flags para reglas.

Plan de pruebas: tests de geometrias frontera; carga 1/2/50 RPS y ráfagas de 5/10/30 min; soak test; caos (Google timeout, GPU indisponible, Service Bus retry, PostgreSQL failover); prueba de restauración; evaluación offline del modelo con dataset de referencia, métricas de precisión acordadas y comparación entre versiones antes de promover.

9. Decisiones arquitectónicas (ADR)

Decisión	Elegida / alternativa y razón
ADR-01 Asincronía	202 + cola frente a HTTP síncrono: menor fragilidad ante 15 s nominales y picos. Ofrecer modo síncrono solo si contrato/SLO y capacidad lo justifican.
ADR-02 Geodatos	PostGIS frente a Cosmos DB genérico: SQL espacial y geometría compleja; benchmark de 150 M obligatorio.
ADR-03 Ejecución AI	Container Apps GPU como candidata frente a AML endpoint; depende de compatibilidad, operativa y throughput probado.
ADR-04 Orquestación	Colas + máquina de estados/outbox frente a cadena acoplada; evita volver a comprar Google/IA después de fallos.
ADR-05 Quota	APIM para control inmediato y ledger independiente para conciliación y facturación.

10. Estimación económica y sensibilidad

No presentaría un precio oficial sin región, SKU, uptime, tamaño físico del índice PostGIS, contrato Google y benchmark GPU. El siguiente rango sirve únicamente como ROM (rough order of magnitude) de discusión, no como cotización ni compromiso. Moneda EUR/mes, impuestos excluidos; costes Google separados. La estimación exige confirmación en Azure Pricing Calculator con suscripción y región.

Componente mensual	Valle — ROM	Pico — ROM
Front Door/WAF + APIM	350–1.200 €	500–1.800 €
Container Apps CPU + Service Bus	100–450 €	350–1.600 €
PostgreSQL/PostGIS (HA/almacenamiento)	650–2.200 €	750–2.800 €
Blob + red + Key Vault	40–220 €	100–500 €
Monitor/Log Analytics/sondas	100–500 €	150–850 €
GPU inferencia (modelo desconocido)	400–4.000 €	2.500–25.000 €+
Subtotal Azure orientativo	1.640–8.570 €	4.350–32.750 €+

Advertencia: los rangos anteriores son hipótesis de presupuesto creadas para priorizar la validación comercial; no proceden de un presupuesto ejecutado en Azure. El coste GPU puede exceder el rango en ráfagas prolongadas, especialmente si la inferencia de 10 s ocupa una GPU completa. El coste de Google Maps no está incluido: estimarlo como número de llamadas por job × tarifa aplicable, con caché solo si autorizada. Añadir backups, contingencia, soporte, salida de datos, HA y reservas si corresponden.

Modelo de costes verificable

GPU ≈ (peticiones procesadas × segundos GPU efectivos por petición ÷ 3.600) × €/hora GPU + tiempo ocioso/cold starts. Google ≈ trabajos × llamadas facturables por trabajo × precio por llamada. Storage ≈ GB-mes acumulados × tarifa + transacciones + egress. Revisar con métricas por cada 1.000 evaluaciones y coste unitario para negociar cuotas/precios.

Optimización: batching si la precisión y latencia lo permiten, imágenes redimensionadas con trazabilidad, evitar recalcular etapas terminadas, lifecycle de blobs, muestreo inteligente de trazas sin perder métricas de negocio, capacidad caliente solo en ventanas previsibles y reducción de mínimos fuera de marzo-mayo.

11. Entrega, responsables y plazos estimados

Fase	Duración indicativa	Criterio de salida
Discovery / PoC	Semanas 1–2	Alineación de SLAs, ingestión muestra catastro, primera latencia GPU y forecast económico.
MVP seguro	Semanas 3–5	API B2B + estado + pipeline + E2E happy path + observabilidad.
Hardening	Semanas 6–8	Carga/chaos, seguridad, Jira, runbooks, recuperación y evaluación del modelo.
Go-live controlado	Semanas 9–10	Canary, revisión SRE/seguridad, costes observados y aceptación contractual.

Equipo base orientativo: 1 arquitecto/technical PM parcial, 1–2 backend .NET, 1 platform/DevOps, 1 ML engineer, QA compartido y acceso a experto de datos/GIS y seguridad. Es una estimación de planificación, no compromiso contractual; ingestión de 150 M, aprobación GPU, licencia externa y calidad del modelo son dependencias de camino crítico.

12. Riesgos prioritarios y mitigaciones

Riesgo	Impacto	Mitigación / decisión
Cuota/capacidad GPU	Cola sin drenar, SLO fallido	Benchmark y solicitud de cuota en semana 1, capacidad alternativa, carga adelantada.
Google Maps indisponible / precio variable	Errores, costes impredecibles	Timeout, retry prudente, circuit breaker, presupuesto por tenant y evaluación de términos.
PostGIS 150 M insuficientemente indexado	p95 inestable, escalado caro	COPY + GiST + EXPLAIN, particionado justificado por benchmark, control de cambios.
Inconsistencia de estados	Dobles cobros, datos perdidos	Outbox, idempotencia, ledger, recuperación y reconciliación periódica.
Fugas cross-tenant / URL pública	Incidente contractual	RLS/filtros, private endpoints, pruebas cruzadas, SAS de vida corta.
Regresión del modelo	Resultados funcionales erróneos	Dataset golden, metric gates, comparación y rollback de versión.

13. Preguntas que plantearía al cliente

• ¿Se acepta integración asíncrona con polling/webhook o existe obligación de respuesta HTTP síncrona?

• ¿Durante cuánto tiempo se mantienen los 50 RPS? ¿Qué p95 E2E y plazo máximo en cola se contratan?

• ¿Qué arquitectura/formato tiene el modelo y cuál es el rendimiento real por T4/A100 en imágenes representativas?

• ¿Qué tamaño físico, geometría media, frescura y reglas de actualización tienen los 150 millones de parcelas?

• ¿Hay obligaciones de residencia del dato, retención/borrado y licencias de imágenes Google?

• ¿Cómo se define el hecho facturable: solicitud aceptada, inferencia ejecutada o resultado final entregado?

• ¿Qué objetivos de disponibilidad, RPO/RTO y cobertura de soporte se requieren?

14. Qué validaría antes de cerrar el diseño

Construiría un test harness reproducible que compare PostGIS con una muestra representativa, mida inferencias en T4/A100 y genere un modelo de costes calibrado por carga real. Validaría la API con dos clientes piloto, haría simulacros de fallo y recuperación, y ajustaría cuotas por tier comercial con datos de uso observados. Solo después cerraría un compromiso fuerte de p95 E2E.

15. Puntos para explicar en la entrevista (12–15 minutos)

Min 0–2: problema y contradicción de carga anual vs RPS. 2–5: explicar secuencia 202 + Service Bus + estados, y qué pasa cuando algo falla. 5–8: PostGIS, GPU, cálculo de backlog y decisión condicionada a benchmark. 8–11: IAM/tenant, quota/facturación y seguridad. 11–13: E2E/Jira, CI/CD y gates. 13–15: ROM de costes, riesgos y próximas preguntas al cliente.

Referencias de diseño y verificación

Microsoft Learn — Azure Container Apps serverless GPUs:
https://learn.microsoft.com/en-us/azure/container-apps/gpu-serverless-overview

Microsoft Learn — API Management quota-by-key:
https://learn.microsoft.com/en-us/azure/api-management/quota-by-key-policy

Microsoft Learn — API Management multitenancy:
https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/service/api-management

Microsoft Learn — PostgreSQL Flexible Server extensions (PostGIS):
https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/concepts-extensions

Microsoft Learn — Application Insights availability tests: https://learn.microsoft.com/en-us/azure/azure-monitor/app/availability

Microsoft Azure Pricing Calculator (para revalidar): https://azure.microsoft.com/en-us/pricing/calculator/

Estado: propuesta arquitectónica para entrevista; no se ha realizado benchmark del modelo ni una cotización en una suscripción real. Las decisiones de capacidad y los SLO deben ratificarse durante discovery.